

Reasoning-first Aligning two traversals of the same route

Songlin Zhang, PhD

The task, in one sentence

We are aligning two side-camera videos recorded on the same loop route, eleven months apart, without GPS or calibration: for every frame of the first drive, which frame of the second shows the same place, and how sure.

What I found when I first looked at the data — before touching any model:

1. **The two drives look very different.** Dry afternoon vs rainy dusk; the second is $2.2\times$ darker with raindrops stuck on the lens; a sideways camera changes with every lane shift.
2. **No sensor to lean on.** No GPS, motion sensor or camera calibration anywhere in the files (checked inside the stream). Only the pictures.
3. **Long stretches that all look the same** — the hardest. Fences, tree tunnels: a frame looks like its neighbours, and like a spot minutes away ($2.4\times$ worse on the right camera).
4. **Some things that look like noise turn out to be systematic:** a slight drift of the right camera's angle, which affects alignment if unhandled; lane changes; speed bumps (slide 17).

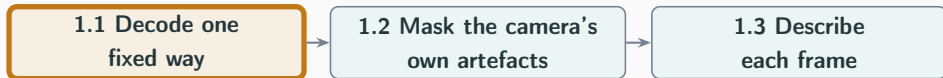
How a person solves it

Nobody recognises a single frame of fence. A person scrubs back and forth and watches what comes *before and after*: the order tells you what one picture cannot. So the algorithm should use the order too, and decide over a stretch of video.

So I framed it as a sequence alignment problem instead of independent image matching.

Stage 1 · 1.1 Decode one fixed way

First, I make the videos reliable before any matching.



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem

Method chosen, and why

Stored order $\neq$ display order for 74.4% of pictures (B-pyramid 0, 4, 2, 1, 3, ...).

I fixed the decoding issue where storage order didn't match display order by **writing a small parser** that rebuilds the picture order from the stream, so frame identity is preserved. (A 1–3-frame scramble is the size of the error being measured.)

No colour matrix declared; pinning one changes 3.17% of decoded bytes.

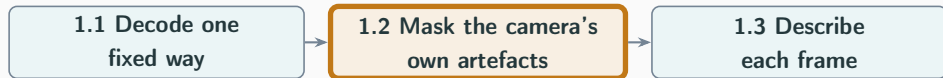
I **standardised colour handling** where the matrix was unspecified (BT.709 pinned, decoded from the original stream), to keep appearance consistent across tools.

Any of the above can **drift silently** when code or settings change.

To prevent silent drifts when configs change, I **bake the data assumptions into the cache key and add integrity checks** (24 gates: 22 pass, 2 waived on record).

Stage 1 · 1.2 Mask the camera's own artefacts

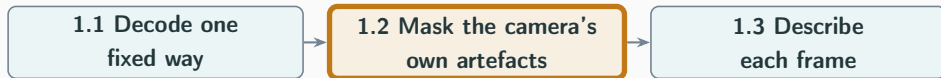
Before extracting descriptors, I mask fixed artefacts like raindrops and mirrors using temporal statistics, with a rule insensitive to the drives' exposure difference.



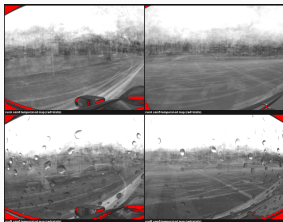
Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem	Method chosen, and why
Raindrops fixed on the lens for the whole rainy drive.	Mask from temporal statistics: staticness = $ \nabla(\text{temporal median}) /(\text{temporal std} + 1)$ with a relative-contrast guard. Camera-attached content is exactly what stays static while the scene moves.
Wing mirror and vignette always in frame.	Same statistic, plus a never-bright test (temporal max < 18% of p99). Dead lens areas never get bright, whatever the scene.
Rainy drive 2.2× darker ; segmenters (ADE20K) have no class for any of this.	No learned model; the ratio is near scale-free, computed per drive at 360×270 over every 8th frame, versioned. Same rule on both drives, no label consumed.

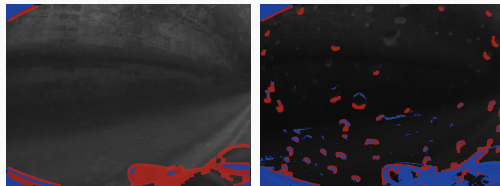
Stage 1 · 1.2 Mask the camera's own artefacts (in pictures)



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision



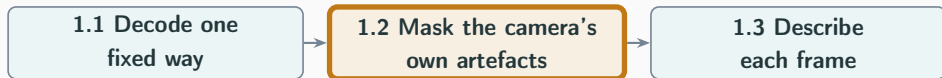
How much each pixel changed over the whole drive. The drops, the mirror and the dark corners never move, which is why a temporal statistic finds them.



The resulting mask on the dry drive (left) and the rainy drive (right): red is camera-attached, blue too dark to use.

Same rule; only the rainy drive grows drops.

Stage 1 · 1.2 Mask the camera's own artefacts: first attempt

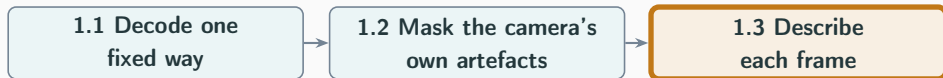


Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

I first tried SegFormer, but it doesn't cover raindrops, mirrors or dark vignetting — which are our main artefacts — so I switched to a temporal-statistics mask.

- Note: on the 8-pair development set SegFormer removed one false accept ($3 \rightarrow 2$) but cut coarse retrieval recall@5 from 0.71 to 0.57. The temporal mask finds the rain where it is: right camera 1.2% (dry) $\rightarrow$ 9.9% (rainy).
- Honest attribution: the mask was rebuilt together with the decode and the dense path, so its effect alone is not isolated in a held-out test.

Stage 1 · 1.3 Describe each frame



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem

Eleven months apart, dry afternoon against rainy dusk.

No labels to train with: the only labels that exist are the blind test set.

Droplets and the mirror would **bias a whole-image descriptor**.

Method chosen, and why

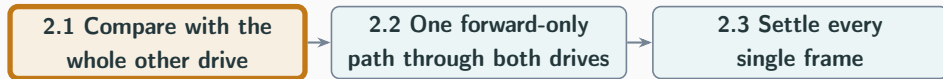
Because the runs are months apart, I start with **frozen DINOv2 patch features and cosine similarity** to get robust coarse matches under lighting change (ViT-S/14 at 448×336 , nothing fine-tuned).

Nothing fine-tuned. Training would either need labels I do not have or spend the test.

Masked mean pooling over reliable patches only. Remove the bias rather than hope the network ignores it. (An unsupervised VLAD pooling over the same tokens is evaluated later as a variant.)

Stage 2 · 2.1 Compare with the whole other drive

The frames can now be trusted. Stage 2 is **sequence matching**, coarse to fine: it finds roughly *where* each frame belongs. This page answers *where do I search?*, the next *how do I find the path?*



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem	Method chosen, and why
At full resolution, matching every frame is too expensive .	So I start with a coarse pass , sampling every fourth frame of both drives.
Where do I search? Look-alike places are sometimes hundreds of frames apart, and the time offset between the drives drifts by around 12 seconds.	A local search band is risky because of that drift, so I compute a full similarity matrix over the whole other drive and keep it cheap by subsampling.

Stage 2 · 2.2 One forward-only path

2.1 Compare with the whole other drive

2.2 One forward-only path through both drives

2.3 Settle every single frame

Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem

How do I find the path? In a fence corridor a single frame carries no identity; a person uses the order of events.

The two drives were **recorded at different speeds**, and the ratio changes along the route.

The route is a **loop**: departure and arrival look alike, so a naive search wraps across the seam.

Method chosen, and why

Dynamic time warping: one path through the similarity matrix that can only move forward. Order is the constraint a person actually uses.

I let the slope vary within measured limits: the bounds come from the drift I measured, not from an assumption.

Departure and arrival are treated as **distinct occurrences**: no wrap, no interpolation across the seam.

Note: step set $\{(1, 1), (1, 2), (2, 1), (1, 3), (3, 1), (2, 2)\}$, slope penalty 0.035, skip penalty 0.008, local slope in $[0.2, 2.0]$.

Stage 2 · 2.2 One forward-only path (in pictures)

2.1 Compare with the whole other drive

2.2 One forward-only path through both drives

2.3 Settle every single frame

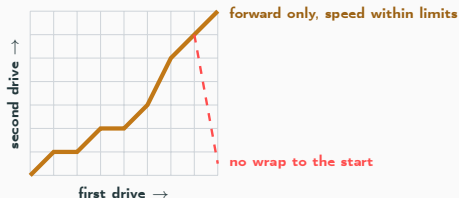
Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision



First and last frames: the car returns to the same bay, same heading. The route is a loop, so its two ends alias.

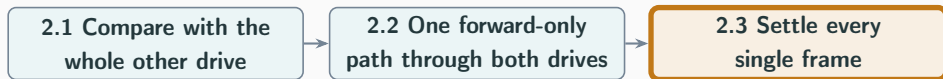


Leaving and returning: the same place at both ends. The path may not jump across this seam.



One path through the similarity grid: it may only move right or up, at a bounded slope, and never back to the start.

Stage 2 · 2.3 Settle every single frame



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem	Method chosen, and why
Stride 4 leaves a ± 2 -frame ambiguity; a consumer needs the exact frame.	Full-cadence refinement: for every frame, argmax of a local structure score within ± 8 of the interpolated coarse path. The coarse path already fixes the neighbourhood, so this is cheap.
The result must stay in sequence order .	Refinement runs under a non-decreasing floor .
A defect: this stage has no speed (slope) prior . Where the local score is flat, the optimiser runs to the window edge, producing implausible jumps — at high confidence.	I reported the limitation as it is. After the freeze I built a Viterbi version with a slope prior that avoids the jumps by construction; because it came after the freeze, it is reference evidence, not the submission.

Stage 2 · 2.3 Settle every single frame (in pictures)

2.1 Compare with the whole other drive

2.2 One forward-only path through both drives

2.3 Settle every single frame

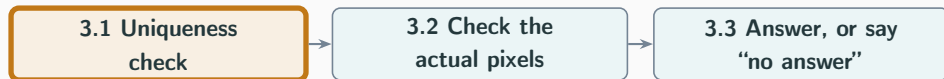
Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision



What this stage's defect looks like: frame 905 of the first drive mapped to frame 802 of the second at top confidence — two different houses. Found by rendering the aligned videos side by side with guide lines, the same habit as looking at the data in the first place.

Stage 3 · 3.1 Uniqueness check

Stage 2 says roughly where I am. Stage 3 checks that the match is unique, confirms the exact frame **geometrically on the pixels**, and decides whether to answer at all.



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem

Some corridors and tree stretches look **almost identical**, so a single frame is easily misjudged.

Repeated scenery comes at **different lengths** — short awnings, long tree tunnels.

What should **confidence** mean here?

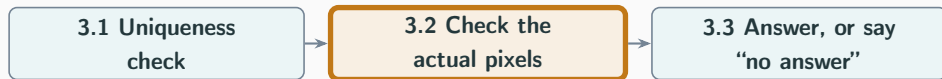
Method chosen, and why

Multi-scale sequence uniqueness: I compare the best match against the second-best, to make sure this stretch is genuinely distinctive.

I check for repeated segments at **several window scales**, using windows of different radii (2 / 5 / 10 / 20 nodes) to confirm uniqueness.

Confidence reflects **uniqueness, not strength**: how big the gap is between the best and the second-best match, not how high the score is. That gap assigns the *strong* and *sequence-supported* tiers.

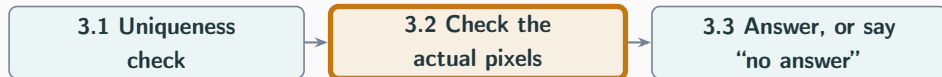
Stage 3 · 3.2 Check the actual pixels



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem	Method chosen, and why
Sequence order says roughly <i>where</i> you are, but foliage and fences still produce false matches with many raw correspondences.	Geometric verification to filter false matches: RootSIFT + ratio test + RANSAC, a pixel-level check on the masked images.
A single textured patch (a tree crown, a fence panel) can mislead the check on its own.	I require support across several grid cells (4×3), so one patch cannot carry the verification.
Which image size to run it at?	896×672 , chosen by a resolution comparison (next page) as the best balance between reliability and cost: at full size the share of consistent matches drops.

Stage 3 · 3.2 Check the actual pixels (the resolution study)



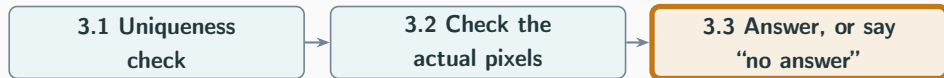
Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

left camera			
Raster	Matches	Inlier ratio	Passing
448 × 336	22	0.8108	2/6
640 × 480	31	0.6957	4/6
896 × 672	48	0.6052	6/6
1440 × 1080	85	0.3859	4/6

right camera			
Raster	Matches	Inlier ratio	Passing
448 × 336	12	0.7076	1/6
640 × 480	21	0.622	1/6
896 × 672	40	0.4629	5/6
1440 × 1080	86	0.3283	2/6

Development pairs only. Passing peaks at 896 × 672 and falls at native, where leaf texture inflates the match count while the inlier ratio collapses.

Stage 3 · 3.3 Answer, or say “no answer”



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision

Problem

After uniqueness and geometry, the system must **decide whether to give a match at all**. Some frames should have no answer: blocked views, very dark stretches, no corresponding segment.

A **wrong answer becomes ground truth** downstream; choosing not to answer only costs coverage.

What is the **confidence** value?

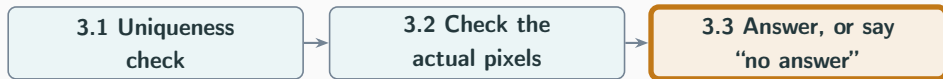
Method chosen, and why

The output carries an **explicit type**: accepted (strong), accepted (sequence-supported), geometry failed, loop ambiguous, out of route, idle segment — each refusal with a clear reason. An empty partner means *no answer under this method*.

So I prefer **no answer over a confident wrong one**, and an empty row is never converted into a negative label.

A **ranking, not a calibrated probability**. I separately implemented an HMM with a null state to get calibrated posteriors; that is reported after the freeze and is not part of the submitted method.

Stage 3 · 3.3 Answer, or say “no answer” (in pictures)



Stage 1 · Prepare the frames → Stage 2 · Sequence matching → Stage 3 · Geometric verification and decision



Rainy drive, left camera, three seconds: a lorry fills the picture. Some places have no usable view, and the method must be allowed to say so. Both methods actually drove through this one at confidence 1.00 — reported later as a failure with its cause.

After the pipeline — Three behaviours that looked like noise (1/2)

Behaviour	How it is handled
The right camera is mounted at a slightly different angle in the second drive. That made me reconsider what a “matching frame” is: <i>same picture</i> and <i>same position of the car</i> are no longer the same frame, so the label has to be based on the real position. The offset is constant along the route (37 px left, 11 px down at the working size; roughly 10–15° of yaw, estimated because there is no calibration), which makes it a fixed correction of about −1.8 frames on this camera.	Matching itself still works, because the pixel check tolerates a shift. For labelling, I pre-shift the second drive by the measured amount, so the annotator judges <i>place</i>, not camera angle. The remaining constant is measured on round 2, confirmed on round 3b (exact-frame precision 0.14 → 0.67), and shipped as a per-frame field (−2 frames) rather than baked into the mapping.

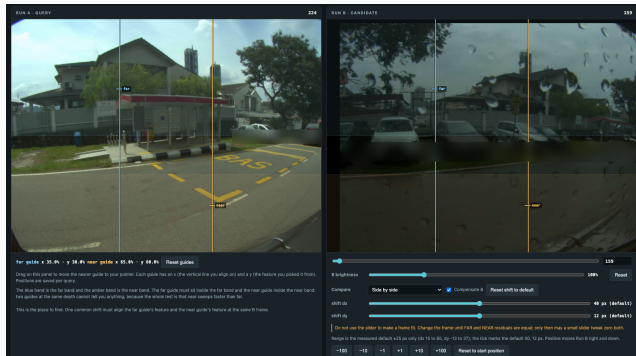
After the pipeline — Three behaviours that looked like noise (2/2)

Behaviour	How it is handled
The car changes lanes. A sideways camera sees nearer objects sweep faster, so a lane change alters the picture more than a few metres of travel does. A near/far test found brief episodes but no lasting lane offset.	Because the path is decided over a stretch of video, a few frames of changed view cannot move it. For labels, the annotator uses the object nearest the image centre, whose position does not depend on lane, and allows ± 2 frames where lanes differ.
Speed bumps jolt the picture. A bump pitches the whole car for a frame or two. Both cameras share one chassis, so a real jolt must show up in both videos at the same frame — which gives a label-free test (raw frames co-fire 0 of 28 times, a corrected signal 4 of 33).	That jolt signal became the motion channel of the probabilistic model, and the same test retired an earlier claim as an artefact. The submitted mapping is unchanged either way (98% / 95% identical rows): the path absorbs a one-frame shake.

Evaluation — How would anyone know it works?

1. **The field checks against GPS:** a hit within 25 m, or ± 10 frames at one frame per second. This data has no GPS, no motion sensor, no calibration and no third drive.
2. **So the only reference for “same place” is a person looking at two frames.** The measuring instrument had to be a person — which means: define “same place” precisely, show the person nothing the method thinks, freeze the method before they start, score once, and test the instrument itself.
3. **Automatic checks are not a substitute.** Do the two cameras agree? Does the loop close? Those only test *consistency*, and things can be consistently wrong — the lorry proved it.

Evaluation — The instrument: a blind labelling page



Query frame from the first drive (left), candidate from the second (right), opened at a plain proportional guess. Three answers: *match* with a tolerance, *cannot tell*, *no such place*. No prediction and no score anywhere on the page.

Three rules that kept the labels consistent

- **Far and near:** one guide on something distant, one on something close. At the right frame a single shift lines up both; near things move faster, so the near one pins the frame.
- **Lane changes:** use the object nearest the image centre.
- **Right camera:** the page pre-shifts the second drive by the measured amount; the slider is not there to make a frame fit.

Evaluation — The contract around the instrument

Step	When (UTC)	What it fixes
Export the query frames	13:03	which frames are asked; fingerprinted
Freeze the method	15:41	every setting; fingerprinted
Annotate	—	40 per camera, frames never used in development
Seal the labels	15:52	label-file fingerprint, citing the freeze
Score once	—	the scorer refuses to run if any setting changed

- **This proves order, not custody:** the method was fixed before the labels were read. It does not prove nobody peeked; that would need a third party, and the report says so.
- **Done three times:** round 2 spread evenly over the route; round 3 aimed at the places round 2 made me suspicious of; round 3b a redo, after the instrument itself failed a test.

Results — Precision at frame tolerances

Method	Cam	Coverage	± 1	± 2	± 5	± 10	exact frame
submitted	left	100%	81%	84%	91%	97%	72%
submitted	right	86.5%	56%	69%	91%	97%	38%
earlier sparse-anchor method	left	53%	82%	88%	100%	100%	65%
earlier sparse-anchor method	right	11%	75%	75%	100%	100%	25%
submitted, targeted round 3	left	100%	71%	81%	86%	100%	62%
submitted, targeted redo 3b	right	91%	38%	71%	86%	100%	14%

- **Errors are small, not gross:** 97–100% land within ± 10 frames. What remains is a few frames off, and on the right camera that has a known cause.
- **Coverage is not accuracy:** the earlier method answered half the left-camera queries and a tenth of the right's, so its precision is on an easier set.

Results — How strict this is, compared with published work

Benchmark	Counts as a hit	Typical top result (2022–24)	Camera
Pitts30k / Tokyo24/7 / MSLS	within 25 m	87–93%	front, rectilinear
Nordland	± 10 frames @ 1 FPS	71–76%	front, rectilinear
Nordland (SALAD / CliqueMining)	± 2 / ± 1 frames	76% / 91%	front, rectilinear
RobotCar Seasons	0.25 m / 5 m	1–15% (SeqSLAM)	side fisheye, rain
This work	± 2 frames $\approx$ 2 m	69–84%	side fisheye, rain
This work	± 10 frames $\approx$ 10 m	97–100%	side fisheye, rain

Not the same data — context for the scale only. Published benchmarks count a hit at 25 m or ten frames; here a frame is one metre, so my ± 2 standard is about ten times tighter, and at the field's own tolerance the result is 97–100%.

Future work, in order

1. **Ship the Viterbi refinement** after a fresh blind round: it removes the jumps by construction.
2. **The learned-vocabulary pooling** as the frame description: the one change that moved the label-free checks.
3. **A learned point matcher** for the rain: the right camera's pixel check finds half as many matches.
4. **Camera calibration** from the recording team: the mount angle is estimated today, not measured.
5. **A “no such place” label set**: the one thing no blind round so far could test.

The rule I kept: no model training, no data leakage

No model was trained on this data, and no label ever entered the pipeline: the method was frozen before each label set was opened, and nothing tuned after a set was read gets that set's number. The submission is the frozen method; everything better is evidence, marked as such.

Lessons learned

1. **Test the instrument, not just the method.** My labelling tool failed twice — once in what “no match” meant, once in a sign error — and I only found out because I treated the labels themselves as data. Both corrections are in the report, with their timing.
2. **Consistency is not correctness.** Two cameras, two methods and every automatic check agreed on the lorry, and all were wrong. A held-out number is worth exactly the process that produced it.
3. **Freeze first, then improve — and keep the two apart.** The Viterbi refinement and the learned-vocabulary pooling are both better than what I submitted. They stay evidence, not the submission, because they came after the labels were opened.

Backup: the exact-frame reading (round 2 labels)

Method	Cam	Accepted	Correct	Precision	95% interval	Coverage	Uncorrected
submitted	left	32/32	23	0.719	0.55–0.84	100%	0.575
submitted	right	32/37	12	0.375	0.23–0.55	86.5%	0.343
earlier sparse-anchor	left	17/32	11	0.647	0.41–0.83	53.1%	0.647
earlier sparse-anchor	right	4/37	1	0.250	0.05–0.70	10.8%	0.250

A hit only inside the annotator's interval (width 0 to ± 1). "Cannot tell" queries leave the denominator; *Uncorrected* keeps them.

Backup: the numbers behind the right camera's constant

Shift applied to the right camera (frames)	0	-1	-2	-3
round 3b, submitted, exact frame	0.14	0.33	0.67	0.57
round 3b, speed-limited variant	0.19	0.43	0.71	0.67
round 3b, probabilistic model	0.17	0.43	0.78	0.65
round 2, right camera (mixed meanings)	0.375	0.34	0.375	0.25
round 3, left camera (no shift, control)	0.62	0.43	0.24	0.10

The constant (-1.79 frames) comes from the round-2 camera diagnostic, built before round 3b's queries were exported; this sweep is a sensitivity reading on spent sets, not a held-out number. The submitted file is unchanged; the constant ships as a per-frame field.

Backup: evidence that costs no labels, and what did not help

Checks that need no labels

- Do the two cameras agree? Over 2,183 frames: typically 1 frame apart, 95% within 11. It flagged the hardest stretch before any label existed.
- Does the loop close? Places seen at the start and again at the end must map to the same place.
- Hide a stretch of the second drive: frames whose partner was hidden truly have none. The left camera notices 40 $\rightarrow$ 84% of them as the hole grows, at 0.7% false alarms; the right camera's detector gets *worse*.

Variants, judged on those checks

- Greyscale, sky mask, a different pair selection, a speed signal, coupling the loop ends: **nothing moves**.
- Changing how the frame description is pooled (a 32-word “vocabulary” learned from the route itself, no labels): 95th-percentile disagreement 11 $\rightarrow$ 8 frames, right-camera coverage 0.82 $\rightarrow$ 0.90. The pooling was the bottleneck, not the input.
- No description change touches the jumps: they belong to the search, not to how frames are described.

Backup: the downstream policy — what a consumer cannot silently misuse

Keep / exclude / quarantine

- **Keep** every decoded image in an archive nobody edits, addressed by frame number plus the exact decoding settings.
- **Exclude** the parked stretches and the blind-test frames; **quarantine** dark, blocked and jump-neighbourhood rows out of the top tier.
- **Never** turn an empty row into a “different places” label; blur plates and faces before anything leaves the archive.

Attached to every frame

- Identity, quality flags, which split it belongs to, and its partner with a typed status and the freeze fingerprint.
- The camera-to-world constant: 0 for the left camera, -2 frames for the right.
- Two facts stated outright: confidence is a rank, not a probability; and many first-drive frames share one second-drive frame, because the second drive was slower.

Backup: what “decode one fixed way” means, and why it was needed

What the files are. Raw H.265 video streams: no container, no index, no timestamps inside. “Frame n ” only exists after decoding.

Four things a decoder gets wrong without telling you

1. **Order.** The encoder stores pictures as 0, 4, 2, 1, 3, $\dots$, so 74.4% sit at a different index than they are shown at. Counting packets as frames scrambles the video by 1–3 frames: the size of the error being measured.
2. **Colour.** Turning the stored signal into RGB needs a colour standard; none is in the file. The decoder guesses from picture height, so the original and a downsized copy decode to different colours: 3.17% of bytes.
3. **Rate.** The file claims both 10 and 25 fps; the logs measure 9.99.
4. **Size and time.** Stored 1440×1088 , shown as 1440×1080 . One timestamp log serves two cameras with different frame counts: a log row is not a frame.

One decoding path, every setting pinned

- Decode from the original file, never from a converted copy.
- A small parser reads the stream, rebuilds the picture order, and must agree with the decoder’s frame count exactly.
- Frame identity = position in the *display* order, from 0; colour standard fixed in the decoding command.
- Every cached result is named after the file fingerprint, size, colour setting and mask version: change any of them and the old cache is not found instead of mixed in.
- 24 automatic checks (fingerprint, frame count, order, size, timestamps, parked bounds): 22 pass, the shared-log check is *waived* with its reason written down.
- Otherwise a decoding mix-up and an alignment error look identical.